LLM-Guided Tool-Aware Task and Motion Planning for Chemistry Lab Automation

Bohyeong Pak[1†], Hyunho Cho[1†], Sanghyun Kim[1,2*]

[1]School of Mechanical Engineering (AgeTech-Service Convergence Major), Kyung Hee University, 1732, Deogyeong-daero, Giheung-gu, Yongin-si, 10587, Gyeonggi-do, Republic of Korea.
[2]Department, Institutes of Convergence Technology, 145, Gwanggyo-ro, Yeongtong-gu, Suwon-si, 16229, Gyeonggi-do, Republic of Korea.

*Corresponding author(s). E-mail(s): kim87@khu.ac.kr;
Contributing authors: bhe1004@khu.ac.kr; chohh7391@gmail.com;
†These authors contributed equally to this work.

Abstract

Autonomous chemistry laboratories can accelerate material discovery and improve experimental reproducibility, yet current systems remain limited in two respects: existing automation pipelines rely on protocol-specific equipment that restricts adaptability to diverse, non-linear workflows, and task and motion planning on robots with limited kinematic redundancy becomes unreliable when strict orientation, collision, and workspace constraints must be satisfied at once. This paper proposes LLM-Guided Tool-Aware TAMP (Task and Motion Planning), a hierarchical framework that combines large language model (LLM)-based symbolic reasoning with GPU-accelerated motion optimization for chemistry automation. The system converts natural-language experimental instructions into validated Chemical Description Language (XDL) protocols, selects appropriate end-effectors and plans obstacle rearrangement based on execution context, estimates the scene state through multi-view fiducial marker-based perception, and produces collision-free, constraint-satisfying trajectories by evaluating thousands of candidates in parallel. A sensor-driven skill library handles execution-level actions such as tool changing and adaptive pouring. On a 6-DoF manipulator, the GPU-parallelized planner achieves a 98.9% planning success rate under orientation constraints across all three tasks, up from 63.3% with a CPU-based sampling baseline, with lower planning-time variance. We build a high-fidelity simulation testbed in *NVIDIA Isaac Sim* for randomized trials and module-level ablation studies, and results confirm that the full framework

executes complete chemical sequences involving tool switching, obstacle rearrangement, and multi-step reagent manipulation. The project page is available at https://rcilab.github.io/auto_chem.

Keywords: Autonomous Chemistry Laboratory, Task and Motion Planning, Large Language Models, Intelligent Planning, GPU-Accelerated Optimization

1 Introduction

Chemistry experiments are the cornerstone of material science discovery and hypothesis verification. Traditionally, these experiments have been conducted manually, a process that is resource-intensive and often limited by human variability in reproducibility. The emergence of Self-Driving Labs (SDLs) aims to automate the experimental cycle—from experiment selection and robotic execution to data acquisition [1, 2]. Despite this progress, existing laboratory automation pipelines typically rely on specialized, single-purpose equipment bound to fixed experimental protocols, requiring manual reconfiguration whenever a new procedure is introduced. This rigidity becomes a critical bottleneck as modern chemical workflows grow increasingly diverse and non-linear: the choice of end-effector depends on the geometry of the target vessel, obstacle rearrangement must account for both the current and subsequent procedural context, and such decisions are difficult to hard-code robustly when the space of possible protocols remains open-ended. Manual domain redefinition for every new tool–task combination does not scale, motivating a system that can reason about execution context and autonomously adapt its tool-use configuration.

Beyond operational flexibility, reliable physical execution remains difficult in laboratory environments. To address this challenge, constrained Task and Motion Planning (TAMP) [3, 4] has been applied to automate multi-step procedures such as pouring, stirring, and reagent transport. For example, Yoshikawa et al. [5] utilized PDDLStream [6] to address spillage prevention through constrained motion planning; however, their work reported that orientation constraints significantly degrade planning success rates on standard robot arms, requiring additional degrees of freedom to recover acceptable performance. This limitation is particularly pronounced in CPU-based sampling planners, which rely on sequentially generating random configurations and projecting them onto the constraint manifold, a process whose success probability decreases as kinematic redundancy is reduced. For 6-DoF manipulators commonly used in collaborative robot environments, the further lack of redundant degrees of freedom exacerbates this degradation in success rates.

To simultaneously address these two challenges—context-aware operational flexibility and constrained planning reliability—this paper proposes the LLM-Guided Tool-Aware TAMP framework. A lightweight LLM serves as a context-aware decision engine that goes beyond converting natural language instructions into executable protocols. It analyzes the physical execution environment at each procedural stage, infers the appropriate end-effector based on object geometry and task requirements, and determines whether and where to rearrange workspace obstacles by jointly considering

the current and next procedure. These decisions are resolved before the planning stage, so that the GPU-accelerated TAMP engine is initialized with an updated configuration and can focus on computing reliable, collision-free trajectories.

To support this execution model, the framework maintains environment state consistency through a robust state grounding interface that fuses multi-view *April-Tag*-based tracking with continuous sensor telemetry, providing the planning engine with consistent world-state estimates. The GPU-accelerated TAMP engine explores thousands of candidate solutions in parallel and converges them toward the constraint manifold through gradient-based optimization, improving planning success rates on 6-DoF manipulators. The main contributions of this paper are as follows:

- A hierarchical decision-making pipeline based on a lightweight LLM is established for context-aware chemistry automation. The pipeline converts natural language instructions into validated XDL protocols, infers the appropriate end-effector from object geometry and task requirements, and determines obstacle rearrangement strategies by jointly analyzing current and subsequent procedural context—enabling dynamic tool switching (e.g., suction, multi-finger grippers) without manual domain redefinition.
- Bounded and predictable motion planning is achieved for constrained 6-DoF laboratory manipulation through GPU-parallelized optimization. Multi-view fiducial marker-based perception provides consistent state grounding under dynamic laboratory conditions, and cuTAMP generates constraint-satisfying, collision-free trajectories within a structurally bounded time budget, mitigating the reliability degradation commonly observed in CPU-based sampling planners on low-redundancy manipulators.
- The proposed framework is implemented as an end-to-end autonomous system and evaluated under randomized initial conditions in a high-fidelity simulation testbed built on *NVIDIA Isaac Sim*. End-to-end performance is validated on long-horizon chemical workflows involving tool switching, obstacle rearrangement, and reagent handling.

The remainder of this article is organized as follows: Section 2 reviews related work in lab automation and TAMP. Section 3 details the proposed framework architecture and its core components. Section 4 presents the experimental results and performance evaluations conducted in a purpose-built simulation testbed based on *NVIDIA Isaac Sim*. Finally, Section 5 offers concluding remarks, and discusses future research directions.

2 Related Work

2.1 Autonomous Chemistry and Self-Driving Laboratories

Self-Driving Labs (SDLs) have emerged as a core technology for accelerating material discovery and ensuring experimental reproducibility. While early SDL research focused on automation tailored to specific experiments, the paradigm has recently shifted toward building flexible laboratories using general-purpose robots. Burger et al. [7] demonstrated autonomous photocatalyst optimization by introducing a mobile robot

into the laboratory, and Fakhruldeen et al. [8] established a modular integration standard for heterogeneous robotic platforms through the ARChemist architecture. Lunt et al. [9] further extended this direction by integrating multiple robots to automate solid-state chemistry workflows.

Despite these advances, existing SDL systems share a common structural limitation: they are largely built around protocol-specific equipment and manually designed execution routines, requiring substantial reconfiguration when new procedures or apparatus are introduced. This rigidity extends to the manipulation layer as well. Kennedy et al. [10] and Huang et al. [11] developed sensor-feedback-based pouring skills, and Yoshikawa et al. [12] automated electrode polishing, yet these efforts assume fixed end-effectors tailored to predefined tasks, and incorporating a new tool typically requires manual domain redefinition rather than autonomous adaptation. Recent efforts have also sought to improve the reliability of individual laboratory instruments through perception-based feedback. Khan et al. [13] integrated a YOLOv8-based computer vision model with the Opentrons OT-2 liquid handling robot to enable real-time detection of pipette tips and liquid volumes, demonstrating that closed-loop quality control can meaningfully improve accuracy in liquid handling tasks. However, this approach remains scoped to a single, purpose-built instrument and a fixed operational domain, and does not address the broader challenge of adapting tool-use configurations across diverse procedural contexts. In summary, while automation of individual experimental procedures has progressed considerably, adaptability to diverse and evolving chemical workflows, where the set of required tools, vessel geometries, and procedural sequences is not known in advance, remains limited.

2.2 Task and Motion Planning for Constrained Laboratory Manipulation

Complex chemistry experiments simultaneously require long-horizon task planning and precise motion planning under strict physical constraints. PDDLStream, proposed by Garrett et al. [6], has become a representative framework for combining symbolic logic with sampling-based planning. Existing studies have applied PDDLStream to enforce geometric constraints such as orientation maintenance during liquid transfer; however, CPU-based sampling planners rely on sequentially generating random configurations and projecting them onto the constraint manifold, a process whose success probability degrades as kinematic redundancy decreases. Furthermore, because the number of sampling iterations required to find a valid solution is inherently stochastic, the termination time of such planners is unpredictable and unbounded, which is a critical limitation for chemistry laboratory automation, where unexpected planning delays can disrupt time-sensitive reaction conditions such as temperature regulation and reagent timing.

Optimization-based motion planning by Muchacho et al. [14] and constraint-satisfying sampling techniques by Kingston et al. [15] have been investigated to improve constraint handling, yet they do not fundamentally resolve the reliability degradation on manipulators with limited degrees of freedom, nor do they guarantee bounded planning latency. Coleman et al. [16] proposed a design to lower the barrier to entry for complex robotic software through a *MoveIt!* case study, but the

underlying challenges of planning reliability and latency predictability remained unaddressed. Hybrid approaches combining optimization-based pre-grasp planning with reinforcement learning have also been explored for dynamic environments. Li et al. [17] proposed a method that integrates quasi-Newton trajectory initialization with a recurrent deep RL policy for dynamic obstacle avoidance and grasping in mobile manipulators. However, learning-based planners of this kind do not provide formal guarantees on constraint satisfaction or planning latency, as convergence depends on stochastic policy training rather than deterministic optimization over a fixed computational budget. Regardless of the underlying approach, whether sampling-based, optimization-based, or learning-based, these methods have primarily been evaluated on general pick-and-place scenarios. Laboratory manipulation poses a compounded difficulty: orientation constraints must be maintained throughout liquid transport, obstacles may need to be rearranged between procedural steps depending on task context, and tool switching must be coordinated within multi-step workflows, all on low-redundancy robots where planning reliability is already degraded. This combination of requirements has received limited attention in existing TAMP literature.

2.3 Large Language Models for Robot Planning and Embodied Decision-Making

The advent of Large Language Models (LLMs) has opened new possibilities for translating abstract natural language commands into robot-executable instructions. Mehr et al. [18] proposed the Chemical Description Language (XDL) standard for the digitization of chemical literature, and Yoshikawa et al. [19] built the CLAIRify system, which uses LLMs to convert chemical texts into robot commands. Darvish et al. (ORGANA) [20] presented an interactive robotic assistant that utilizes an LLM-based planner to converse with scientists and assist with experiments, and ChemCrow [21] and Coscientist [22] demonstrated that LLMs can autonomously search for chemical tools and design experimental procedures.

While these systems demonstrate strong capabilities in semantic decomposition and high-level task sequencing, their use of LLMs generally remains confined to text-based procedure generation or human decision-making assistance. The generated plans are typically handed off to a separate, manually conFig. d execution pipeline, and the LLM itself is not required to reason about the physical execution environment, such as which end-effector is geometrically appropriate for a given vessel, or whether workspace obstacles must be rearranged before a procedure can proceed. As a consequence, geometric validity, collision avoidance, and orientation feasibility are not addressed at the reasoning stage and must be delegated entirely to downstream planning modules. Recent work has sought to close this gap through simulation-based feasibility verification. Qu et al. [23] proposed TRTP, a three-stage framework that couples vision-language models with digital twin simulation to evaluate spatial reachability and physical feasibility of LLM-generated action sequences before execution. While this closed-loop architecture improves plan reliability, the feasibility check is performed in a separate simulation module rather than being resolved at the symbolic reasoning stage; end-effector selection and context-dependent obstacle rearrangement

remain outside the scope of the language model's decision-making. This separation means that existing LLM-based robot planning systems can formulate what to do, but lack the capability to reason about how it should be physically executed in a given workspace configuration.

2.4 Position of This Work

Existing work has addressed individual aspects of the autonomous chemistry from complementary perspectives. SDL and chemistry automation systems have demonstrated automated experimental execution, but remain largely protocol-specific and assume fixed tool configurations that require manual redesign for new procedures. Constrained TAMP approaches have tackled motion planning under geometric constraints, but exhibit reliability degradation on low-redundancy manipulators and do not guarantee bounded and predictable planning latency for time-sensitive chemical workflows. LLM-based robot planning systems have enabled high-level semantic reasoning for chemical procedures, but do not extend to physical execution grounding or context-dependent execution decisions such as tool selection and obstacle rearrangement. Limited attention has been paid to jointly addressing context-aware symbolic reasoning for end-effector selection, obstacle rearrangement, and procedural sequencing together with reliable, bounded-time constrained execution on general-purpose laboratory manipulators. The proposed framework is positioned at the intersection of these three directions, unifying LLM-based context-aware decision-making with GPU-accelerated constrained planning within a single end-to-end system for flexible and reliable chemistry laboratory automation.

3 Methods

Section 2 identified three coupled limitations that prevent existing approaches from achieving flexible and reliable chemistry laboratory automation: fixed SDL pipelines require manual redesign when procedures or apparatus change; CPU-based sampling planners suffer from degraded reliability and unbounded latency on low-redundancy manipulators under orientation constraints, and LLM-based robot planning systems can often infer what to do but lack the capability to reason how to execute it physically in a specific workspace. To address these limitations within a unified pipeline, the proposed framework is organized as a four-layer hierarchy, where each layer resolves a distinct class of failure before passing control to the next. The LLM Module handles semantic and context-level failures by generating validated protocols and reasoning about tool selection and obstacle rearrangement; the Perception Module ensures consistent world-state grounding through multi-view fiducial tracking and continuous sensor feedback; the TAMP Module provides bounded-time motion planning through massively parallel GPU-based optimization; and the Skill Library Module improves execution robustness by translating nominal plans into sensor-feedback-driven motion primitives. The following subsections first introduce the overall system architecture and then describe each module in detail.

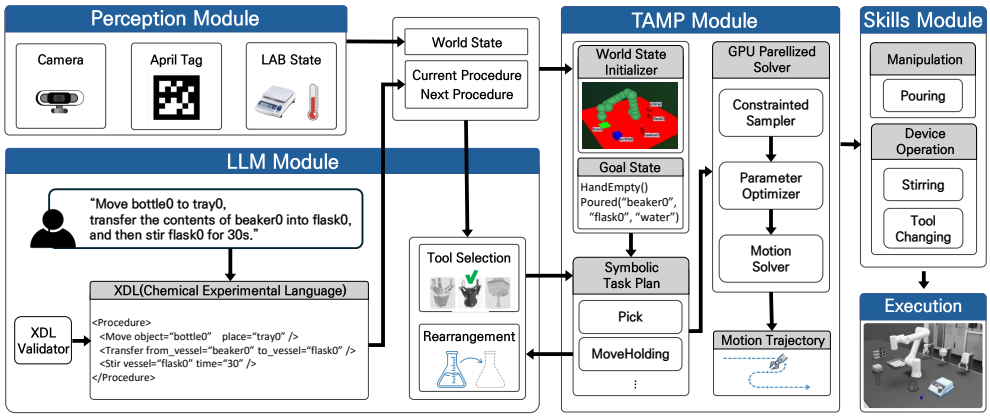

Fig. 1 Architectural overview of the LLM-Guided Tool-Aware TAMP framework. The hierarchy illustrates the integration of symbolic reasoning (LLM) with physical execution (TAMP) supported by a multi-modal perception interface and a robotic skill library.

3.1 Framework Overview

The proposed LLM-Guided Tool-Aware TAMP framework translates a single natural-language command into the complete robotic execution of multi-step chemical experiments. The system is organized as a four-layer hierarchy in which each layer resolves a distinct class of failure before passing its output to the next, allowing downstream modules to operate under progressively tighter guarantees. Fig. 1 illustrates the overall architecture and inter-module data flow. The pipeline begins with the LLM Module, which resolves semantic and context-level failures. It converts the natural-language instruction into a validated XDL protocol and infers an execution context, including tool selection and the need for obstacle rearrangement. To improve reliability, the LLM is restricted to discrete symbolic decisions, while precise spatial computation is delegated to the planning engine. The Perception Module resolves state-grounding failures by providing consistent 6-DoF localization of laboratory assets and aggregating sensor telemetry. Its outputs support protocol validation, world-state initialization, collision-model updates, and execution feedback.

The TAMP Module resolves planning-reliability failures. Using the execution context from the LLM Module and the updated world state from the Perception Module, it computes collision-free, constraint-satisfying trajectories through GPU-parallelized optimization, yielding bounded planning latency and improved reliability on low-redundancy manipulators. The Skill Library Module resolves execution-level failures that arise from physical effects not explicitly modeled during planning. It converts nominal trajectories into sensor-feedback-driven motion primitives for task execution. This separation is deliberate: the TAMP Module ensures geometric and kinematic feasibility of the nominal plan, while the Skill Library handles runtime uncertainties related to contact, flow, and device interaction. Together, these layers prevent semantic, perceptual, kinematic, and physical failures from cascading across module boundaries.

3.2 LLM Module

The LLM Module serves as a context-aware decision engine that converts natural language instructions into validated XDL protocols and determines the appropriate end-effector and obstacle rearrangement strategy before physical execution begins. To meet the real-world engineering constraints of a Self-Driving Lab (SDL), the module is built on a lightweight edge-computing architecture rather than relying on cloud-based large models. The overall pipeline consists of three stages: global experimental protocol synthesis via the XDL Generator, grammar and physical feasibility validation via the XDL Validator, and environment-aware tool and auxiliary action reasoning via the Action Reasoner.

3.2.1 Global Experimental Protocol Synthesis

The XDL Generator functions as a deterministic parameter extraction engine that directly maps natural language commands to validated XDL protocols. Unlike general-purpose large models, which tend to produce conversational responses and suffer from numerical hallucinations during complex procedures, the module employs a lightweight Llama 3.2 1B model fine-tuned via LoRA-based supervised fine-tuning (SFT) on a domain-specific dataset of 5,000 manually constructed samples. The dataset covers synonym variations for the six core XDL operators, continuous numerical parameter ranges, and diverse object instances; detailed composition and training specifics are described in Section 4.1.3. By delegating symbolic extraction to this fine-tuned model, the framework mitigates semantic hallucinations at the generation stage and allows the TAMP module to focus exclusively on physical interactions and geometric trajectory optimization.

3.2.2 Grammar and physical feasibility validation

Despite the high conversion accuracy of the XDL Generator, a protocol that is syntactically correct and semantically coherent may still be physically infeasible. The XDL Validator addresses this by enforcing hardware and physical constraints across three layers before any robot motion occurs. The first, syntactic validation, checks that the output is well-formed XML and that all tags correspond to permitted XDL operators, blocking case-sensitivity mismatches and undefined tags. The second, object existence validation, cross-references all referenced objects against the workspace inventory reported by the Perception Module, blocking commands that target hallucinated objects. The third, physical feasibility validation, sequentially simulates the content state of each vessel throughout the procedure, rejecting sequences that violate physical preconditions, such as stirring an empty vessel or transferring from an empty source. When a conflict is detected at any layer, execution is halted and the user is notified of the error type and the offending step, enabling targeted revision before any robot motion occurs. This feedback-based mechanism ensures that physical contradictions are resolved at the language level rather than at the execution level, preventing hardware damage and experimental failures.

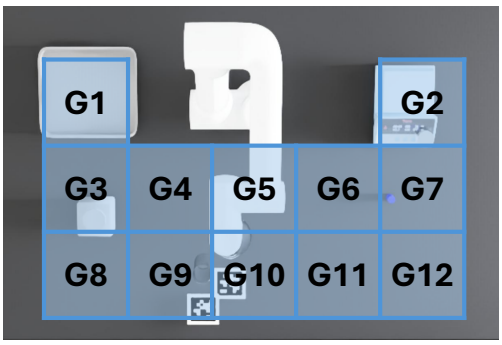

Fig. 2 Discretized grid representation of the workspace used by the ActionReasoner. The workspace is divided into 12 grids (G1–G12), where the ActionReasoner specifies only the target grid category and precise placement within the grid is determined by the TAMP module through collision checking.

3.2.3 Context-Aware Tool and Action Reasoning

Even with a validated protocol and a known world state, execution can fail if the workspace is cluttered or the wrong end-effector is mounted. The Action Reasoner addresses this by determining, prior to each procedure, which end-effector to use and whether workspace obstacles must be rearranged, thereby preventing the TAMP module from entering deadlock states. Rather than performing continuous 3D spatial reasoning directly, the module converts the workspace state received from the Perception Module into a discretized grid representation, as illustrated in Fig. 2, and leverages the pattern-matching capability of a fine-tuned Llama 1B model. Given the current procedure (Current XDL), the subsequent procedure (Next XDL), obstacle information in current environment, and the grid-based spatial state as input, the model infers the required end-effector for the current procedure, whether preliminary object relocation is needed, the auxiliary tool for relocation, and the relocation destination (Target Grid ID).

This design provides two key advantages. First, semantic tool assignment: by grounding the pre-trained model's knowledge of object geometry and affordances onto the physical constraints of the mounted end-effectors, the module assigns appropriate grippers even for vessel types unseen during training, enabling tool switching without manual domain redefinition. Second, context-aware relocation: by jointly analyzing the current and next procedure, the module preemptively relocates obstacles to the work zone required by the subsequent step, eliminating redundant rearrangement in later procedures. Since auxiliary relocation does not require high positional precision, the LLM specifies only the target grid category, and precise placement within that grid is determined by the TAMP module through collision checking. This division of responsibility keeps the LLM from performing infeasible continuous 3D reasoning while maintaining a tractable TAMP search space, and ensures that the TAMP module is always initialized with a workspace configuration from which a valid trajectory can be found.

3.3 Perception Module

The perception module serves as the primary interface for world-state grounding, providing the Task and Motion Planning (TAMP) engine with real-time 6-DoF pose information of laboratory assets. In an autonomous chemistry environment, precise and reliable localization of laboratory apparatus including beakers and flasks is critical for ensuring safe and collision-free manipulation. The module therefore adopts a fiducial marker-based strategy to achieve robust pose estimation at low computational latency. *AprilTag* markers are rigidly affixed to each vessel, and their poses are continuously estimated from two spatially diverse, calibrated RGB cameras mounted around the workspace.

3.3.1 Lab Object Localization

Each target object is assigned a unique *AprilTag* identifier, and two spatially diverse cameras independently report marker poses via the ROS 2 TF tree. Their observations are subsequently fused to produce a single reliable estimate, improving robustness against occlusions and distance-dependent projection errors.

Single-Camera Pose Computation: For each camera $c \in \{1, 2\}$, the pose of object i in the robot base frame is obtained by chaining two TF lookups, where ${}^{\mathrm{b}}T_c$ is the fixed extrinsic calibration of camera c and ${}^{c}T_{\mathrm{t}}^{(i)}$ is the marker pose reported by the *AprilTag* detector. The Euclidean distance from camera c to the marker is also recorded to assess observation reliability:

$$ {}^{\mathrm{b}}T_{\mathrm{t},c}^{(i)} = {}^{\mathrm{b}}T_c \cdot {}^{c}T_{\mathrm{t}}^{(i)}, \tag{1} $$

$$ d_c^{(i)} = \left\| \mathbf{t}\left({}^{c}T_{\mathrm{t}}^{(i)}\right) \right\|_2. \tag{2} $$

Inverse-Distance Weighted Fusion: When both cameras yield valid observations, the two estimates are fused by assigning higher confidence to the camera closer to the marker, thereby minimizing distance-dependent projection errors. A regularization constant $\varepsilon = 10^{-6}$ is introduced to prevent division by zero. The fused translation is computed as the inverse-distance weighted average, and the fused orientation is obtained via spherical linear interpolation (SLERP) between the two unit quaternions:

$$ w_c = \frac{1}{(d_c^{(i)})^2 + \varepsilon}, \qquad W_c = \frac{w_c}{w_1 + w_2}, \tag{3} $$

$$ \tilde{\mathbf{t}} = W_1\, \mathbf{t}_1 + W_2\, \mathbf{t}_2, \tag{4} $$

$$ \tilde{\mathbf{q}} = \mathrm{SLERP}(\mathbf{q}_1,\, \mathbf{q}_2,\, W_2). \tag{5} $$

If occlusion occurs and only one camera provides a valid observation, its estimate is used directly without fusion.

Tag-to-Object Offset Transform: Since the *AprilTag* marker is affixed to the side surface of the vessel rather than its geometric center, a fixed rigid offset ${}^{\mathrm{t}}T_{\mathrm{o}}$ is

applied to recover the true physical center of the object, where ${}^{\mathrm{t}}T_{\mathrm{o}}$ encodes the predefined translation from the marker's local coordinate frame to the vessel's volumetric center:

$$ {}^{\mathrm{b}}T_{\mathrm{o}}^{(i)} = {}^{\mathrm{b}}\tilde{T}_{\mathrm{t}}^{(i)} \cdot {}^{\mathrm{t}}T_{\mathrm{o}}. \tag{6} $$

The resulting transform is broadcast over the ROS 2 TF tree at 30 Hz. This continuous stream forms the foundational World State, which is subsequently consumed by the LLM module for context-aware reasoning and the TAMP module for collision-free trajectory generation.

3.3.2 State Grounding and World Modeling

The World State is constructed from the 6-DoF poses $(x, y, z, \mathrm{roll}, \mathrm{pitch}, \mathrm{yaw})$ provided by the Perception Module at the beginning of each XDL procedure and updated after every stage, ensuring that the planning module always reflects the current laboratory configuration resulting from the execution of the previous stage.

To construct the World State, the module distinguishes between target objects to be manipulated in the current stage and surrounding objects that are not. All non-target objects are defined as obstacles and included in the planning environment to enable collision-free motion planning. Elements with fixed locations, such as tables and laboratory equipment, are maintained as static obstacles, while movable objects such as beakers, bottles, and flasks are continuously updated via the Perception Module.

To reduce planning overhead and the computational burden of collision checking, object models passed to the TAMP module are represented using simplified geometric shapes rather than full CAD geometries. Target objects are approximated as axis-aligned boxes or cylinders, while non-target objects are modeled as slightly enlarged simple geometric obstacles. This abstraction improves the efficiency of collision checking and path sampling while preserving execution feasibility, ensuring that planning overhead remains tractable as the number of laboratory objects grows.

3.3.3 Multi-Modal Sensor Stream Processing

Beyond geometric world-state grounding, the perception module acts as a centralized telemetry hub that aggregates, filters, and distributes continuous data streams from various physical sensors. This real-time sensor processing is critical for closing the control loop in the Skill Library during physical execution, such as in adaptive pouring or temperature regulation. The module primarily processes two sensory modalities:

Scale Data: For precise quantitative liquid transfer, the module interfaces with an electronic precision scale. The raw weight measurements are acquired, synchronized with the system clock, and processed into a stable data stream. This filtered scale data is directly consumed by the Adaptive Pouring skill, serving as the primary feedback variable for the PD controller to dynamically adjust the robot's wrist angle in real-time.

Temperature Data: To ensure strict adherence to chemical reaction conditions, the module also integrates temperature readings from external laboratory equipment, such as hotplate thermocouples. Raw temperature readings are monitored and smoothed to reject transient electrical noise. This provides a stable thermal state

stream, which the device operation skills utilize to verify reaction temperatures and, if necessary, suspend task execution until the target thermal conditions are rigorously met.

By consolidating and filtering these heterogeneous data streams, the perception module shields the high-level planning and execution modules from hardware-specific noise, guaranteeing a clean and synchronized sensory interface for complex chemical manipulation.

3.4 Task and Motion Planning with Adaptive Tool Switching

The TAMP Module computes collision-free, constraint-compliant trajectories by combining a GPU-accelerated planning backend with chemistry-specific operators, and integrates the execution conditions determined by the LLM Module to adaptively switch tools and insert auxiliary actions as needed. Each XDL procedure is treated as an independent planning unit, with the world state updated by the Perception Module used as the initial condition. The module is described across two subsections: the core planning engine and its chemistry-specific operators in Section 3.4.1, and the LLM-guided tool switching and auxiliary action insertion mechanism in Section 3.4.2.

3.4.1 Task and Motion Planning Module

CPU-based sampling planners exhibit reliability degradation under orientation constraints on low-redundancy manipulators, as established in Section 2.2, making them insufficient for the constrained manipulation requirements of chemistry laboratory automation. The planning module addresses this by building on cuTAMP, a GPU-accelerated TAMP backend that formulates planning as a parallelized differentiable optimization problem, simultaneously refining thousands of candidate particles toward the constraint manifold via gradient-based optimization. Unlike CPU-based planners whose termination time is inherently stochastic, cuTAMP executes a fixed number of optimization steps over a fixed batch of particles, making its runtime structurally bounded and predictable, a property that is critical in chemistry laboratory automation where unpredictable planning delays can disrupt time-sensitive reaction conditions such as temperature control and reagent timing. The initial state for each stage is constructed from the world state updated by the Perception Module, and all objects other than the target are treated as obstacles, establishing collision-free plans that reflect environmental changes from previous stages. To support the manipulation requirements of the chemistry experiment domain, the following chemistry-specific operators are defined in addition to the base TAMP operators.

SidePick is designed for grasping the lateral surface of cylindrical vessels such as beakers and flasks. Unlike conventional top grasps, a side grasp sampling constraint is applied that generates candidates approaching from all directions around the object. In the object coordinate frame, where the z-axis is aligned with the vertical axis of the cylindrical vessel and the origin is located at the geometric center of the object, a grasp pose $\mathbf{g} = (\mathbf{t}, \mathbf{r})$ consists of translation $\mathbf{t} \in \mathbb{R}^3$ and orientation $\mathbf{r} = (r, p, y)$. To enable full-circumference side grasping of cylindrical objects, the approach angle α is

uniformly sampled in the horizontal plane:

$$\alpha \sim \mathcal{U}(0, 2\pi). \tag{7}$$

The grasp orientation is determined as follows:

$$r = 0, \quad p = \alpha, \quad y \in \left\{-\frac{\pi}{2}, -\frac{\pi}{2.3}, \frac{\pi}{2.3}, \frac{\pi}{2}\right\}, \tag{8}$$

where roll r is fixed to zero such that the gripper x-axis aligns with the vertical direction, pitch p determines the approach direction around the object circumference via the approach angle α so that the gripper z-axis (finger approach direction) points horizontally toward the object center, and yaw y is discretely sampled from gripper tilt candidates. The grasp height along the object z-axis is sampled within a range centered at the object origin:

$$t_z \sim \mathcal{U}\left(-\frac{h_o}{3}, \frac{h_o}{3}\right), \tag{9}$$

where the lateral position in the horizontal plane is determined by the approach direction α at a fixed radial offset from the object surface. This full-circumference sampling strategy improves search efficiency during particle initialization and enables stable grasping of rotationally symmetric vessels from any direction.

MoveHolding moves the robot while grasping an object and enforces an upright constraint throughout the entire trajectory to prevent spillage during transportation of liquid-filled vessels. The opening direction vector $\mathbf{u}_o$ of the grasped object is derived from the robot joint variables $\mathbf{q}$ and the fixed grasp transform $\mathbf{R}_{grasp}$ determined at the time of grasping:

$$\mathbf{u}_o(\mathbf{q}) = \mathbf{R}_{ee}(\mathbf{q}) \cdot \mathbf{R}_{grasp}^{-1} \cdot \mathbf{z}_w, \tag{10}$$

where $\mathbf{R}_{ee}(\mathbf{q})$ is the end-effector rotation matrix obtained from forward kinematics and $\mathbf{z}_w = [0, 0, 1]^T$. Since $\mathbf{R}_{grasp}^{-1} \cdot \mathbf{z}_w$ is a constant vector computed once at the time of grasping, this constraint can be evaluated solely from the robot joint variables without requiring real-time object pose estimation. The alignment error between this vector and the world z-axis is formulated as an upright cost J_{up}, which equals zero when the object is aligned with the vertical axis and increases as the object tilts:

$$J_{up}(\mathbf{q}) = 1 - \mathbf{u}_o(\mathbf{q}) \cdot \mathbf{z}_w. \tag{11}$$

When $J_{up} = 0$, the object is perfectly upright. For a maximum allowable tilt angle θ_{max}, the constraint is satisfied when:

$$J_{up}(\mathbf{q}) \leq 1 - \cos(\theta_{max}), \tag{12}$$

which constraint is equally applied to the Place operator.

Pour plans the tilting motion after the robot reaches the top of the target vessel, while transportation is handled by MoveHolding. The TAMP module generates

a geometric trajectory that tilts the opening of the vessel toward the target vessel, ensuring the starting pose and collision avoidance for the pouring action. This trajectory serves as the initial condition for the Adaptive Pouring skill in the Skill Library, which dynamically adjusts the wrist angular velocity through PD control using real-time weight feedback from an electronic scale until the target amount is reached. Upon completion of the Pour operator, the state of the corresponding vessel transitions to Poured.

Stir activates the Device Operation skill from the Skill Library to send the target RPM and stirring duration to the magnetic stirrer, without involving motion planning. The stirrer operates at the specified RPM for the set duration, and upon completion, the state of the vessel transitions to Stirred. This operator reflects the design principle that device-level operations are handled entirely at the execution layer, keeping the symbolic planning domain focused on geometric and logical sequencing.

If task planning or motion planning fails during any XDL procedure, up to three retries are performed. Upon task planning retry, the plan skeleton search is re-executed with new initial conditions; upon motion planning retry, particles are re-initialized and optimization is performed again. If a valid solution is not found after three retries, the system halts the workflow and returns an error report containing the failure cause and the relevant procedure information, allowing the user to diagnose and revise the experimental protocol.

3.4.2 LLM-Guided Adaptive Tool Switching and Auxiliary Action Insertion

Even with a reliable planning engine, execution can fail if the mounted tool is incompatible with the current task or if workspace obstacles block the planned trajectory. The Action Reasoner, described in Section 3.2.3, addresses this by processing the current procedure, the subsequent procedure, and the updated environmental state prior to each XDL procedure, producing four inference outputs that are integrated into the TAMP execution pipeline as follows.

Tool changes are treated as pre-execution steps independent of the TAMP symbolic domain, avoiding domain redefinition overhead each time hardware changes. When required, cuTAMP generates a collision-free trajectory to the tool exchange point, after which the Tool Changing skill completes the physical exchange via electromagnet control and mechanical latching.

When rearrangement is required, an auxiliary Pick-MoveHolding-Place sequence is inserted before the main procedure, with a tool change to the auxiliary tool performed first if necessary. Rearrangement targets are specified only at grid-category level by the LLM, with precise placement determined by cuTAMP's collision checking. As described in Section 3.2.3, the target zone is selected to reflect the context of the subsequent procedure, eliminating redundant rearrangement in later steps. The stage-wise world-state update before and after each procedure ensures that plans reflect all prior changes, including object positions, tool configuration, and auxiliary rearrangement outcomes. This entire mechanism is summarized in Algorithm 1.

Algorithm 1 LLM-Guided Adaptive TAMP with Tool Switching

Require: XDL Procedure X_i, Next Procedure X_{i+1}, World State S, Current Tool T_{curr}

Ensure: Executed Motion Trajectory τ

$[T_{req}, \mathrm{f}_{re}, T_{aux}, G_{target}] \Leftarrow$ LLM_Reasoning(X_i, X_{i+1}, S) ▷ Infer tool, rearrangement, rearrangement tool, and target grid
if f_{re} is **true then** ▷ Check if workspace needs reconfiguration
  if $T_{curr} \neq T_{aux}$ **then**
    Execute_Tool_Change$(T_{curr} \rightarrow T_{aux})$
    $T_{curr} \Leftarrow T_{aux}$
  end if
  $S \Leftarrow$ *Update_World_State*() ▷ Ground perception before rearrangement
  $\tau_{aux} \Leftarrow$ *GPU_TAMP_Planner*(Pick-Move-Place, S, G_{target}) ▷ Plan relocation to target grid
  Execute(τ_{aux})
end if
if $T_{curr} \neq T_{req}$ **then** ▷ Validate tool for primary experimental task
  Execute_Tool_Change$(T_{curr} \rightarrow T_{req})$
end if
$S \Leftarrow$ *Update_World_State*() ▷ Final state update for main task
$\tau_{final} \Leftarrow$ *GPU_TAMP_Planner*(X_i, S, T_{req}) ▷ Plan main task trajectory
return *Execute*(τ_{final})

3.5 Skill Library Module and Execution

The TAMP module constructs plans by considering only geometric constraints and logical sequencing; however, physical execution introduces variables that cannot be handled symbolically at the planning stage, such as the hydrodynamic properties of fluids and the response latency of laboratory equipment. The Skill Library addresses this gap by providing a collection of sensor-feedback-based action primitives that refine and instantiate the nominal trajectories generated by TAMP at execution time. While the TAMP module generates plans using standard operators in a skill-agnostic manner, the Skill Library closes the gap between plan and reality by invoking the appropriate skill mapped to each operator.

3.5.1 Manipulation Skills

Manipulation skills enhance the precision and success rate of operations by augmenting the nominal trajectories planned by TAMP with real-time sensor feedback at execution time.

Adaptive Pouring. The TAMP module plans the Pour task as a geometric trajectory for tilting the vessel. Upon reaching the pouring configuration, however, the execution module overrides this nominal trajectory with a corrective trajectory calculated via real-time feedback from a precision scale. Specifically, the skill dynamically adjusts the wrist angle and oscillation velocity using a shaping function and a PD controller based on the error between the target and current weight, compensating for

unpredictable fluid flows that are difficult to model at the planning stage and enabling precise quantitative transfer.

Automatic Tool Changing. The TAMP module plans the tool changing task solely as a geometric motion to the tool rack, without accounting for the electronic control elements required for physical coupling and decoupling. Upon the robot reaching the exchange point, the Tool Changing skill is activated, transforming the abstract replace command into a concrete hardware operation by applying control signals to the electromagnet and executing the mechanical latching sequence, thereby completing the physical exchange.

3.5.2 Device Operation Skills

The TAMP module treats operations such as heating or stirring as abstract symbols. Device operation skills translate these symbols into specific communication protocols and feedback loops at runtime to control heterogeneous laboratory devices.

Feedback-Based Device Control. When the execution module receives a plan containing Heat or Stir commands, it invokes the corresponding skill to communicate with the hotplate or stirrer. Beyond merely activating the device, the skill continuously monitors internal sensor values such as temperature or RPM, and suspends execution or performs local feedback control until the specific physical conditions are strictly met, even if the TAMP plan implies an immediate transition to the next step. This ensures that experimental conditions are verified at the execution layer rather than assumed at the planning layer, preventing reaction failures caused by premature progression.

4 Experiments and Evaluation

4.1 Experiment Setup

4.1.1 Hardware and Simulation Environments

The proposed framework is evaluated on a 6-DOF collaborative manipulator, the *FAIRINO-FR5*, selected for its 5 kg payload capacity and 922 mm working radius, which together provide sufficient workspace coverage for complex reagent handling and apparatus relocation in a laboratory setting. To demonstrate tool-aware manipulation across geometrically diverse laboratory objects, the manipulator is equipped with three end-effectors, each chosen for its mechanical advantages in a specific subset of tasks.

The 2-finger parallel gripper (*AG95*) serves as the default tool for general-purpose manipulation of standardized test tubes and small lab assets. The 3-finger adaptive gripper (*DH3*) is integrated for tasks with high geometric complexity, as its three-point contact geometry enables stable top-down picking of large-diameter or cylindrical vessels such as beakers and wide-neck flasks, while also simplifying the TAMP search space by providing higher axial alignment tolerance compared to side-grasping methods. The suction gripper (*VGC10*) is employed for objects with significant flat surface areas, such as container lids and storage boxes, reflecting common industry practice in chemical laboratory automation.

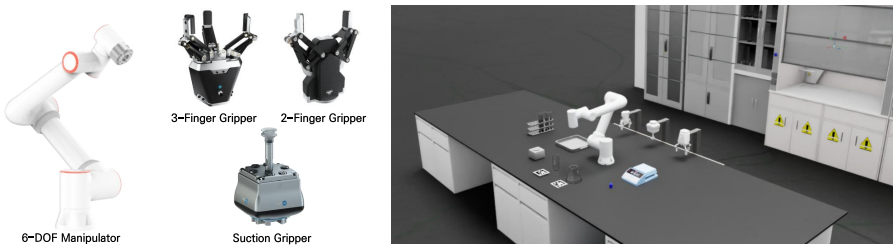

Fig. 3 Hardware components and simulation environment for experimental evaluation. (Left) The 6-DOF collaborative manipulator (*FAIRINO-FR5*) alongside three specialized end-effectors: the 2-finger parallel gripper, 3-finger adaptive gripper, and suction gripper. (Right) The automated chemistry laboratory setup within the *NVIDIA Isaac Sim* environment.

Evaluating the full framework under controlled and reproducible conditions in a physical laboratory would require prohibitively many trials. To address this, a high-fidelity simulation testbed was developed within NVIDIA Isaac Sim, which supports large-scale randomized trials by generating 30 distinct initial configurations per task under strictly identical constraint settings, and allows controlled ablation by isolating individual modules while holding all other variables constant. The TAMP world state is synchronized with the simulation by extracting 6-DOF object poses and joint configurations at each stage, and the GPU-optimized trajectories are streamed back to the simulation for execution, ensuring consistent state grounding and adaptive planning throughout sequential manipulation.

4.1.2 Chemical Experiments

Three core manipulation tasks are selected as representative evaluation scenarios, chosen to cover the essential procedures of real-world chemical experiments while spanning a range of motion planning complexity and manipulation difficulty. **Move** represents physical workspace reconfiguration, such as handling equipment covers or storage boxes, and is intentionally designed to involve large-radius objects that challenge the grasp capabilities and kinematic limits of standard grippers, rather than evaluating simple pick-and-place of small objects. **Transfer** represents liquid handling, a fundamental procedure in chemical synthesis, and tests the system's ability to maintain spill-free orientation constraints throughout the transit phase while tilting the source container and pouring its contents into a target vessel. **Stir** is chosen over simpler device operations such as heating because it demands a significantly higher level of spatial precision: the robot must not only relocate the vessel onto a magnetic stirrer but also precisely insert a magnetic stir bar into the narrow opening, making it an ideal scenario for evaluating complex, multi-step task and motion planning.

4.1.3 LLM Fine-Tuning and Training Details

The two LLM components introduced in Section 3.2, the XDL Generator and the Action Reasoner, are each fine-tuned on a domain-specific dataset constructed to reflect the combinatorial structure of the target task.

Table 1 Overall Performance of Natural Language to XDL Generation

Metric	**Result** (%)
Total Trials	100
Valid	97
Invalid	3
Success Rate	97

XDL Generator. The training dataset is constructed at a scale of 5,000 samples across four combinatorial axes: synonym variations for each of the six core XDL operators (Add, Stir, HeatChill, Transfer, CleanVessel, Move), continuous numerical parameter ranges (0–20 mL for volume, 0–20°C for temperature, 1–30 s for time), four instance identifiers per vessel type yielding 16 unique object identifiers, and command complexity ranging from single-operator instructions to three-step composite procedures with a weighted distribution of 30% single-step, 40% two-step, and 30% three-step. Since the task constitutes a structural mapping to a fixed XDL schema, the output space is inherently constrained, and this dataset sufficiently covers the principal combinatorial patterns, as empirically supported by the 97% conversion success rate reported in Section 4.2.1. Training is performed by applying LoRA (rank $r = 16$, $\alpha = 32$, dropout 0.05) to a 4-bit quantized Llama 3.2 1B model, using an 8-bit AdamW optimizer with a learning rate of 2×10^{-4}, an effective batch size of 16, and 5 epochs.

Action Reasoner. The training dataset consists of 3,000 samples generated combinatorially by placing up to four objects on 12 discretized workspace grids. For each combination of the six XDL operators, relocation necessity is determined according to task-specific blocking criteria, and the ground-truth destination is assigned as either a context-aware target zone or a workspace edge depending on whether the obstacle is the target of the subsequent procedure. Candidate grids are provided as input features after excluding fixed equipment and occupied grids, categorized solely by locational characteristics to prevent label leakage. The dataset is balanced across rearrangement-required and rearrangement-unnecessary cases, as well as context-aware and irrelevant obstacle cases, to prevent over-prediction of relocation. Training follows the same Llama 3.2 1B base model with LoRA conFig. d at rank $r = 8$, $\alpha = 16$, dropout 0.05, a learning rate of 1×10^{-4}, batch size 16, and 3 epochs.

4.2 XDL Generation Evaluation

4.2.1 Language Command to XDL Format

Valid XDL protocols were generated in 97 out of 100 trials, with no syntax errors or semantic tag confusion observed across any test case, as summarized in Table 1. To rigorously evaluate performance under varying task complexities, the 100 test instructions span single-step commands to three-step composite workflows, with a weighted distribution of 30% single-step, 40% two-step, and 30% three-step procedures, incorporating randomized synonym combinations and complex sentence structures beyond isolated commands. The three failures all occurred at the physical feasibility layer and originated from contradictions in the user instructions themselves, as discussed in

Table 2 Classification of Injected XDL Errors for Validator Benchmarking

Validation Layer	Error Category	Error Condition	Injected XDL Example	Remarks
Syntax	Missing required attribute	Intentional omission of mandatory attributes	`<Transfer to_vessel="flask_A" volume="10 mL" />`	from_vessel missing
Syntax	Undefined tag / attribute name	Use of disallowed tag or attribute name	`<Move object="flask_A" to_vessel="flask_B" />`	Using to_vessel instead of place in Move
Object existence	Non-existent object reference	Reference to object absent from workspace	`<Stir vessel="flask_C" time="30 s" />`	flask_C not registered in workspace
Physical feasibility	Physically infeasible sequence	Physical operation on an empty container	`<Stir vessel="beaker_A" time="1 min" />`	Stirring empty beaker_A

Section 4.2.2. The average inference time per command was 1.57 seconds, confirming that the module introduces negligible latency relative to the overall pipeline execution time.

4.2.2 XDL Validator

The **XDL Validator** achieves 100% classification accuracy on the benchmark set, with zero false positives and zero false negatives, confirming that it provides complete coverage of the error space defined by the three-layer validation structure. Of the 100 generation trials, all three detected errors occurred at the physical feasibility layer: immediately after CleanVessel transitioned a vessel to an empty state, a subsequent Stir operator required a filled state, triggering an execution halt. In each case, the system returned a targeted error message, such as "Cannot Stir empty vessel: flask A," enabling the user to revise the instruction before any robot motion occurred. To further verify the robustness of the validator, a benchmark test set was constructed by injecting anomalous data across four error categories corresponding to the three-layer validation structure, as summarized in Table 2. The resulting confusion matrix in Table 3 confirms that the validator reliably intercepts all injected errors regardless of error type, demonstrating that the three-layer structure provides complete and non-overlapping coverage of the physically relevant failure modes.

4.3 TAMP Planning Reliability and Latency Evaluation

The proposed planning module achieves higher planning success rates than the CPU-based baseline across all three chemical tasks, with the most pronounced improvement on the Transfer task where orientation constraints are most restrictive: 96.7% versus

Table 3 XDL Validator Classification Accuracy (Confusion Matrix)

	Validator: Pass (Positive)	Validator: Fail (Negative)
Valid	TP = 20	FN = 0
Invalid	FP = 0	TN = 80

Table 4 Comparison of planning success rate and time between CPU-based (PDDLStream) and GPU-accelerated (cuTAMP) TAMP solvers on the 6-DoF *FR5* manipulator. Planning time reports the mean ± std of successful trials only. Timeout budget: 35 s per trial. Each condition was evaluated over 30 randomized trials.

Task	Solver	Success (%)	Planning Time (s)	Timeout (%)
Transfer	PDDLStream	63.3	5.63 ± 7.63	20.0
	cuTAMP	96.7	30.93 ± 0.53	3.3
Move	PDDLStream	83.3	0.17 ± 0.12	0.0
	cuTAMP	100.0	17.60 ± 3.55	0.0
Stir	PDDLStream	60.0	0.79 ± 0.70	26.7
	cuTAMP	100.0	28.83 ± 1.00	0.0

63.3% on the same 6-DoF manipulator. Beyond success rate, the proposed planning module exhibits structurally bounded and predictable runtime across all tasks, a property that CPU-based sampling planners cannot guarantee. These results are summarized in Table 4.

Both solvers were evaluated on 30 randomized initial configurations per task under identical starting poses, target conditions, orientation constraints for spillage prevention, and a wall-clock time budget of 35 seconds per trial. The GPU-accelerated planner was conFig. d with 1,024 parallel particles and 1,000 optimization steps per planning attempt. The reported metrics isolate algorithmic planning latency only, strictly excluding physical execution and simulation rendering times.

On the Transfer task, the proposed planning module achieves 96.7% success by simultaneously optimizing thousands of candidate particles toward the constraint manifold via gradient-based optimization, enabling effective exploration of the feasible region even when kinematic redundancy is insufficient. PDDLStream, by contrast, achieves only 63.3%, with 20.0% of trials timing out, as its sequential sampling mechanism depends heavily on whether early random configurations happen to satisfy the orientation constraint. On the Move task, the lower constraint complexity of top-grasp-based manipulation allows PDDLStream to reach 83.3%, yet the proposed planning module still eliminates all remaining failures to achieve 100.0%. On the Stir task, the requirement for precise stir-bar insertion into narrow vessel openings imposes tight spatial constraints that reduce PDDLStream to 60.0% with 26.7% timeouts, while the proposed planning module maintains 100.0%.

Table 5 LLM Tool Selection and Corresponding Performance

Task	Selected Tool	Target Object	Success Rate (%)
Move	***DH3***	Beaker, Flask, Tube	100.0%
	VGC10	Bottle, Box	97.3%
Transfer	***AG95***	Beaker, Flask, Tube	100.0%
Stir	***DH3***	Beaker, Flask, Tube	96.67%

The temporal predictability of the two approaches differs as fundamentally as their success rates. On the Transfer task, the CPU-based solver exhibits high variance among successful trials (std = 7.63 s), while the proposed planning module maintains a narrow time band (std = 0.53 s). This contrast reflects the bimodal outcome distribution of PDDLStream: trials where early samples resolve constraints terminate quickly, while trials where they do not cannot recover within the time budget and are excluded as failures, producing an artificially low standard deviation that masks the underlying unpredictability. The proposed planning module, by contrast, succeeds on all 30 trials for the Move and Stir tasks; its higher standard deviation on these tasks reflects the inclusion of configurations that PDDLStream could not solve at all. Overall, the proposed planning module is the only solver that guarantees every trial terminates with a valid solution within a bounded time envelope, a property that is critical for time-sensitive chemical workflows where unpredictable planning delays risk compromising reaction conditions.

4.4 LLM-based Tool Selection and Rearrangement Action Reasoning

The LLM module achieves 100% accuracy on tool selection, rearrangement judgment, and auxiliary tool selection across 600 unseen test scenarios, and accurately assigns appropriate end-effectors even for object types not present in the fine-tuning dataset. These results confirm that the module generalizes reliably beyond its training distribution, enabling tool switching and context-aware obstacle rearrangement without manual domain redefinition.

4.4.1 Semantic Tool Assignment and ActionReasoner Evaluation

Tool selection accuracy reaches 100% for Move, Transfer, and cylindrical-vessel Stir scenarios, and 97.3% for large flat-surface objects such as bottles and boxes, as summarized in Table 5. These results are obtained on 100 test scenarios constructed to include unseen object types not present in the fine-tuning data, confirming that the module generalizes in a zero-shot manner by combining its pre-trained knowledge of object geometry and affordances with the physical constraints of the mounted end-effectors. The model consistently assigns the *DH3* gripper for cylindrical vessels across Move and Stir tasks, the *AG95* gripper for Transfer to enable side-grasp pouring, and the *VGC10* suction gripper for large flat-surface objects that exceed the graspable width of finger grippers, including unseen object types such as bottles and boxes.

Table 6 ActionReasoner Inference Accuracy on Unseen Test Scenarios

Metric	Accuracy (%)
Main Tool	100.0
Need Rearrange	100.0
Rearrangement Tool	100.0
Target Grid (category)	100.0
Target Grid (exact)	76.5

The average inference time of 0.0665 seconds confirms that tool selection introduces negligible latency in the overall pipeline.

These results also demonstrate that no single fixed-gripper configuration can cover the full range of task–object combinations encountered in chemistry laboratory environments. Using *AG95* alone prevents grasping of large objects such as bottles and boxes due to grip width limitations. Using *DH3* alone prevents the side-grasp-based Pour action required for liquid transfer. Using *VGC10* alone makes stable grasping and precise insertion of cylindrical vessels infeasible. LLM-based dynamic tool assignment is therefore not an optional enhancement but a prerequisite for operational coverage across diverse laboratory workflows.

The comprehensive reasoning capability of the Action Reasoner is evaluated on 600 unseen test scenarios covering all four inference outputs: tool selection, rearrangement necessity, auxiliary tool, and destination grid. As summarized in Table 6, 100% accuracy is achieved for the first three outputs. For the destination grid, category-level accuracy reaches 100%, while the exact grid match rate within a category is 76.5%. This discrepancy is by design: the LLM specifies only the target grid category, and the precise placement within that category is delegated to the TAMP module through collision checking, as described in Section 3.2.3. The category-level accuracy of 100% therefore confirms that the Action Reasoner correctly identifies the intended work zone in every scenario, which is the decision that matters for downstream planning reliability.

4.4.2 Context-Aware Rearrangement Strategy Evaluation

Context-aware rearrangement achieves the highest sequence completion rate across both evaluated sequences, reaching 96.67% on Transfer → Stir and 83.33% on Stir → Transfer, compared to 50.0% and 40.0% for random rearrangement and 0% and 13.33% for no rearrangement. These results are obtained by comparing three conditions, all evaluated with LLM-based tool selection identically enabled to isolate rearrangement strategy as the independent variable. The No Rearrange condition executes the main task directly even when obstacles block the path. The Random Rearrange condition detects obstacles and relocates them to arbitrarily selected unoccupied grids, serving as an ablation baseline that isolates the spatial clearance effect

Table 7 TAMP Success Rates and Sequence Completion Rates by Rearrangement Strategy

Sequence	Metric	No Rearrange	Random Rearrange	Context-Aware
Transfer → Stir	Step 1 Success	0%	60.0%	96.67%
	Sequence Completion	0%	50.0%	96.67%
Stir → Transfer	Step 1 Success	100.0%	100.0%	100.0%
	Sequence Completion	13.33%	40.0%	83.33%

Table 8 End-to-End Pipeline Success Rates for Long-Horizon Chemical Workflows

Experimental Workflow	**XDL Gen. Success (%)**	**Planning Success (%)**	**Execution Success (%)**	**End-to-End Success (%)**
Move → Transfer	100.0	100.0	100.0	100.0
Transfer → Stir → Move	100.0	96.67	96.67	96.67
Move → Transfer → Stir	100.0	96.67	96.67	96.67
Overall Average	**100.0**	**97.78**	**97.78**	**97.78**

from the context-awareness effect. The Context-Aware condition uses the LLM to analyze the subsequent procedure and preemptively relocates obstacles to the work zone required by the next step.

Table 7 shows the Step 1 TAMP success rate and sequence completion rate for each condition across both sequences. The Transfer → Stir sequence is evaluated because rearrangement is most frequent due to the wide workspace required by side-grasp pouring, and Stir → Transfer is included as a reverse-order control to confirm that results are not order-dependent. When rearrangement is disabled, Step 1 success drops to 0% on Transfer → Stir due to workspace congestion. Random rearrangement recovers Step 1 success to 60.0% by clearing the immediate workspace, but since obstacles are placed without regard to the subsequent procedure, Step 2 requires additional rearrangement or encounters new collisions, limiting sequence completion to 50.0%. Context-aware rearrangement simultaneously secures workspace for Step 1 and positions obstacles where they are needed for Step 2, achieving 96.67% sequence completion and eliminating the redundant manipulations that random rearrangement incurs. This confirms that context-aware obstacle placement, rather than mere spatial clearance, is the key factor in maintaining reliable execution across multi-step chemical workflows. Fig. 4 illustrates the resulting execution flow for a representative Transfer procedure, showing how the auxiliary relocation sequence is inserted prior to the main task when rearrangement is required.

4.5 End-to-End System Evaluation

The integrated framework achieves an overall average end-to-end success rate of 97.77% across three long-horizon chemical workflows, as summarized in Table 8. A trial is counted as successful only when every procedural step completes in order; any

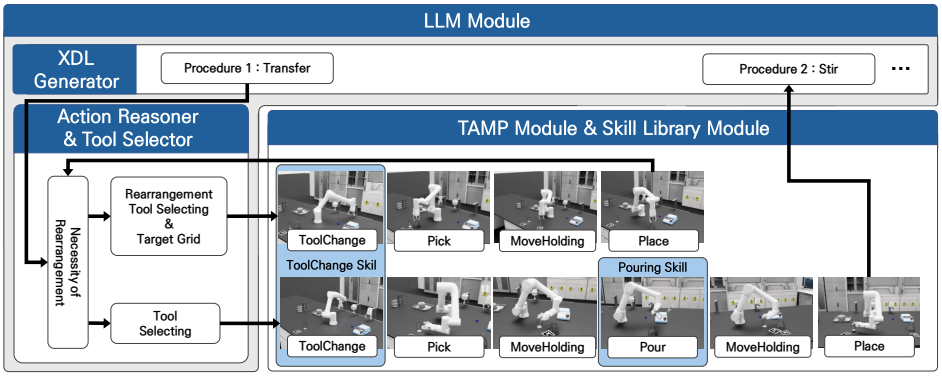

Fig. 4 Execution flow of the TAMP pipeline for a Transfer procedure. The ActionReasoner determines tool selection and rearrangement necessity. When rearrangement is required, an auxiliary relocation sequence (upper path) is inserted prior to the main experimental procedure (lower path), which includes sensor-feedback-based skills such as adaptive pouring.

failure at an intermediate step is treated as a complete trial failure. Each workflow is evaluated over 30 independent trials initialized with randomized world states and robot configurations, ensuring that results are not biased toward specific laboratory layouts.

The evaluation decomposes end-to-end performance into three sequential stages to identify potential bottlenecks: XDL generation, in which the LLM correctly translates the natural language command into a validated executable protocol; planning, in which the GPU-accelerated TAMP module finds a collision-free kinematic trajectory augmented with LLM tool-selection and rearrangement reasoning; and execution, in which the simulated robot physically completes the motion plans without constraint violations such as collisions, liquid spillage, or grasp failures. As shown in Table 8, XDL generation succeeds in 100% of trials across all three workflows. Planning and execution each achieve 96.66% on the two three-step workflows, with the Move → Transfer sequence reaching 100% across all stages. The consistent alignment between planning and execution success rates confirms that the Skill Library reliably instantiates the planned trajectories under real-world execution variables, introducing no additional failure modes beyond those already captured at the planning stage.

Representative execution snapshots of the Move → Transfer → Stir workflow are illustrated in Fig. 5, showing the robot successfully completing all three procedural stages across a single trial. These results confirm that the proposed framework, by unifying LLM-based context-aware decision-making, GPU-accelerated constrained planning, and sensor-feedback-based skill execution within a single pipeline, enables reliable autonomous execution of complex multi-step chemical workflows from a single natural language command.

5 Conclusion

This paper addressed two core limitations of existing chemistry laboratory automation systems: unreliable constrained motion planning on low-redundancy manipulators,

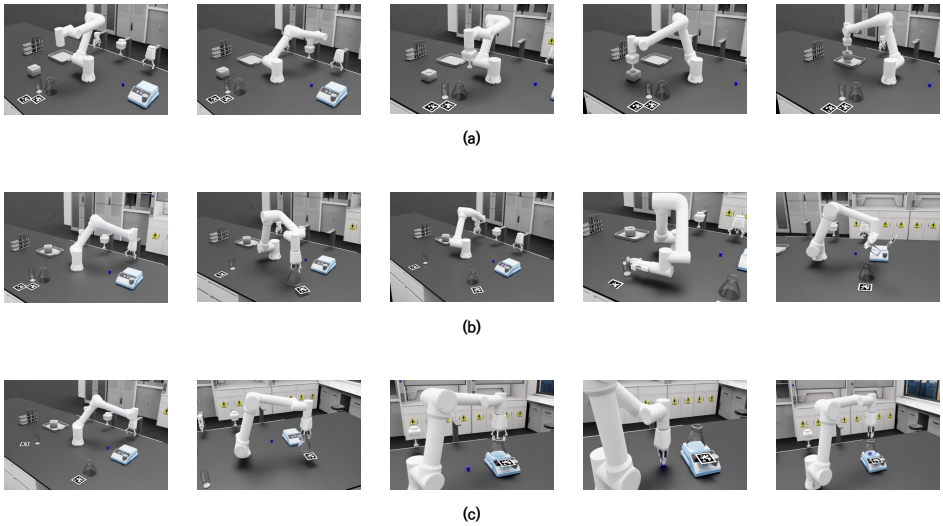

Fig. 5 Sequential execution snapshots of the **Move** → **Transfer** → **Stir** workflow evaluated in the end-to-end experiments. Each row shows five representative keyframes captured during a single trial. (a) **Move**: the robot relocates the container to the target zone. (b) **Transfer**: liquid reagent is poured into the destination vessel. (c) **Stir**: the mixture is stirred on the hotplate to complete the reaction sequence.

and insufficient operational flexibility due to reliance on fixed tool configurations. To resolve both simultaneously, we proposed the LLM-Guided Tool-Aware TAMP framework, which unifies LLM-based context-aware decision-making with GPU-accelerated constrained planning within a single end-to-end pipeline.

The framework resolves the planning reliability limitation by replacing CPU-based sampling with GPU-parallelized gradient optimization, improving the planning success rate from 63.3% to 96.7% on the Transfer task where orientation constraints are most restrictive, while also guaranteeing structurally bounded runtime that enables reliable temporal coordination in time-sensitive chemical workflows. The operational flexibility limitation is resolved through LLM-based dynamic tool assignment and context-aware obstacle rearrangement, which accurately assign appropriate end-effectors even for unseen object types and preemptively position obstacles to eliminate redundant manipulations across sequential procedures. Together, these two capabilities enable the integrated framework to achieve a 97.77% end-to-end success rate on long-horizon chemical workflows initiated from a single natural language command.

Real-robot deployment remains to be demonstrated, as the primary gap lies in sim-to-real transfer due to differences in hydrodynamic properties and real-world sensor noise. Several architectural choices are designed to minimize this gap, including the ROS 2-native architecture, the *AprilTag*-based perception pipeline, and the sensor-feedback-driven Skill Library, such that the primary integration effort consists of collision-model calibration and sensor-interface commissioning rather than architectural redesign. Future work will address this through physical deployment on the FR5 platform, alongside expansion of the supported XDL operators and skill

library to cover more complex chemical procedures such as titration, filtration, and centrifugation.

Declarations

Funding

add data access later

Conflict of Interest The authors have no conflicts of interest to disclose that relate to the subject matter of this article

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

[1] Seifrid, M., Pollice, R., Aguilar-Granda, A., Morgan Chan, Z., Hotta, K., Ser, C.T., Vestfrid, J., Wu, T.C., Aspuru-Guzik, A.: Autonomous chemical experiments: Challenges and perspectives on establishing a self-driving lab. Accounts of Chemical Research **55**(17), 2454–2466 (2022)

[2] Abolhasani, M., Kumacheva, E.: The rise of self-driving labs in chemical and materials sciences. Nature Synthesis **2**(6), 483–492 (2023)

[3] Garrett, C.R., Chitnis, R., Holladay, R., Kim, B., Silver, T., Kaelbling, L.P., Lozano-Pérez, T.: Integrated task and motion planning. Annual review of control, robotics, and autonomous systems **4**(1), 265–293 (2021)

[4] Wang, Z., Garrett, C.R., Kaelbling, L.P., Lozano-Pérez, T.: Learning compositional models of robot skills for task and motion planning. The International Journal of Robotics Research **40**(6-7), 866–894 (2021)

[5] Yoshikawa, N., Li, A.Z., Darvish, K., Zhao, Y., Xu, H., Kuramshin, A., Aspuru-Guzik, A., Garg, A., Shkurti, F.: Chemistry lab automation via constrained task and motion planning. arXiv preprint arXiv:2212.09672 (2022)

[6] Garrett, C.R., Lozano-Pérez, T., Kaelbling, L.P.: Pddlstream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In: Proceedings of the International Conference on Automated Planning and Scheduling, vol. 30, pp. 440–448 (2020)

[7] Burger, B., Maffettone, P.M., Gusev, V.V., Aitchison, C.M., Bai, Y., Wang, X., Li, X., Alston, B.M., Li, B., Clowes, R., *et al.*: A mobile robotic chemist. Nature **583**(7815), 237–241 (2020)

[8] Fakhruldeen, H., Pizzuto, G., Glowacki, J., Cooper, A.I.: Archemist: Autonomous robotic chemistry system architecture. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 6013–6019 (2022). IEEE

[9] Lunt, A.M., Fakhruldeen, H., Pizzuto, G., Longley, L., White, A., Rankin, N., Clowes, R., Alston, B., Gigli, L., Day, G.M., *et al.*: Modular, multi-robot integration of laboratories: an autonomous workflow for solid-state chemistry. Chemical Science **15**(7), 2456–2463 (2024)

[10] Kennedy, M., Schmeckpeper, K., Thakur, D., Jiang, C., Kumar, V., Daniilidis, K.: Autonomous precision pouring from unknown containers. IEEE Robotics and Automation Letters **4**(3), 2317–2324 (2019)

[11] Huang, Y., Wilches, J., Sun, Y.: Robot gaining accurate pouring skills through self-supervised learning and generalization. Robotics and Autonomous Systems **136**, 103692 (2021)

[12] Yoshikawa, N., Akkoc, G.D., Pablo-García, S., Cao, Y., Hao, H., Aspuru-Guzik, A.: Does one need to polish electrodes in an eight pattern? automation provides the answer. Digital Discovery **4**(2), 326–330 (2025)

[13] Khan, S.U., Møller, V.K., Frandsen, R.J.N., Mansourvar, M.: Real-time ai-driven quality control for laboratory automation: a novel computer vision solution for the opentrons ot-2 liquid handling robot: Su khan et al. Applied Intelligence **55**(7), 524 (2025)

[14] Muchacho, R.I.C., Laha, R., Figueredo, L.F., Haddadin, S.: A solution to slosh-free robot trajectory optimization. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 223–230 (2022). IEEE

[15] Kingston, Z., Moll, M., Kavraki, L.E.: Sampling-based methods for motion planning with constraints. Annual review of control, robotics, and autonomous systems **1**(1), 159–185 (2018)

[16] Coleman, D., Sucan, I., Chitta, S., Correll, N.: Reducing the barrier to entry of complex robotic software: A moveit! case study. arxiv 2014. arXiv preprint arXiv:1404.3785

[17] Li, Y., Ke, L., Zhang, C.: Dynamic obstacle avoidance and grasping planning for mobile robotic arm in complex environment based on improved td3: Y. li et al. Applied Intelligence **55**(11), 776 (2025)

[18] Mehr, S.H.M., Craven, M., Leonov, A.I., Keenan, G., Cronin, L.: A universal system for digitization and automatic execution of the chemical synthesis literature. Science **370**(6512), 101–108 (2020)

[19] Yoshikawa, N., Skreta, M., Darvish, K., Arellano-Rubach, S., Ji, Z., Bjørn Kristensen, L., Li, A.Z., Zhao, Y., Xu, H., Kuramshin, A., *et al.*: Large language models for chemistry robotics. Autonomous Robots **47**(8), 1057–1086 (2023)

[20] Darvish, K., Skreta, M., Zhao, Y., Yoshikawa, N., Som, S., Bogdanovic, M., Cao, Y., Hao, H., Xu, H., Aspuru-Guzik, A., et al.: Organa: A robotic assistant for automated chemistry experimentation and characterization. Matter **8**(2) (2025)

[21] Bran, A.M., Cox, S., Schilter, O., Baldassari, C., White, A.D., Schwaller, P.: Chemcrow: Augmenting large-language models with chemistry tools. arXiv preprint arXiv:2304.05376 (2023)

[22] Boiko, D.A., MacKnight, R., Kline, B., Gomes, G.: Autonomous chemical research with large language models. Nature **624**(7992), 570–578 (2023)

[23] Qu, Y., Hu, X., Chen, F., Wei, Z.: Trtp: a three-stage robust task planning framework for open worlds via visual-language models and digital twin simulation. Applied Intelligence **56**(4), 107 (2026)